Übungsblatt 1: Bare-Metal, Polling und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser ersten Laborübung machen Sie sich mit der Entwicklungsumgebung und der Hardware vertraut. Sie lernen Grenzen und Einsatzbedingungen von Polling-Architekturen (Super-Loops) kennen — insbesondere das *Heavy-Load-Problem* bei langen unzerlegten Abschnitten. Anschließend verlagern Sie die periodische LED-Aktion in einen Timer-Interrupt (Foreground/Background). Hardware: **Board NUCLEO-H755ZI-Q** (MB1363), **MCU STM32H755ZIT6**, Arbeit auf dem Cortex-M4.

Aufgabe 1: Hello World (LED Blink)

Um die Toolchain zu überprüfen, erstellen wir ein einfaches Bare-Metal-Programm auf dem Cortex-M4-Kern.

1. Legen Sie ein neues Projekt für das Board **NUCLEO-H755ZI-Q** (MCU STM32H755ZIT6) in der STM32CubeIDE an.
2. Konfigurieren Sie die gelbe LED **LD2 an PE1** (MB1363) als `GPIO_Output` und weisen Sie sie dem Cortex-M4 zu (*Pin Context Assignment*). Boardrevision/Bestückung bei Bedarf in UM2408 prüfen.
3. Generieren Sie den Code; prüfen Sie GPIO- und Clock-Initialisierung.
4. Lassen Sie die LED in der `main.c` des CM4-Projekts toggeln (`HAL_Delay(500)`). Hinweis: Toggle alle 500 ms ergibt Flankenabstand 500 ms, aber einen vollständigen On-Off-Zyklus von $\approx 1\,\text{s}$ (1 Hz).

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Code nur in `USER CODE`-Blöcken; die generierte `while(1)` nicht durch eine zweite Endlosschleife im USER-Block ersetzen.

```
/* USER CODE BEGIN 3 - Inhalt der vorhandenen while(1) */
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1); /* LD2 gelb */
HAL_Delay(500); /* blockiert; kompletter LED-Zyklus   1 s */
/* USER CODE END 3 */
```

Aufgabe 2: Das Polling-Desaster

Wir provozieren einen Timing-Fehler, der Grenzen einer Super-Loop bei langer Hintergrundarbeit zeigt. Polling ist nicht grundsätzlich ungeeignet — lange unzerlegte Arbeit kann aber das Zeitbudget sprengen.

1. Schreiben Sie eine *Heavy-Load*-Funktion (Zählschleife), deren Arbeit der Compiler nicht entfernt (Parameter/`volatile`, Ergebnis ggf. nach außen nutzen). Iterationen sind **keine** bekannte Millisekunden-Dauer; Buildoptionen notieren.
2. Rufen Sie die Funktion in der vorhandenen `while(1)` direkt vor dem LED-Toggle auf.
3. Beobachten Sie die LED. Unterscheiden Sie: **konstante** Last verlängert den Abstand konstant (noch kein Periodenjitter); für Schwankungen bewusst **variable** Lasten einsetzen.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Hilfsfunktion (z. B. `USER CODE BEGIN 0`; Parameter `volatile`, damit die Schleife nicht wegoptimiert wird):

```
void Heavy_Load(volatile uint32_t n)
{
    while (n-- > 0) {
        /* absichtlich leer: reine CPU-Last */
    }
}
```

```
/* USER CODE BEGIN 3 - in vorhandener while(1) */
Heavy_Load(800000); /* Iterationen   bekannte ms-Dauer */
HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
HAL_Delay(500);
/* USER CODE END 3 */
```

Erklärung: In der Super-Loop liegen Last und Toggle sequenziell. Die Laufzeit von `Heavy_Load()` addiert sich zum Delay — der sichtbare Takt weicht vom Soll ab. Bei konstanter Last ist das zunächst eine Periodenverschiebung; Jitter entsteht erst bei schwankender Laufzeit (Optimierung, Interrupts, variable n).

Aufgabe 3: Rettung durch den Timer

Wir stellen periodische Flanken per Hardware-Timer wieder her — unter dokumentierten Bedingungen messbar, aber ohne automatischen harten Fristnachweis.

1. Aktivieren Sie für den CM4 im Device Configuration Tool **TIM2**, *Clock Source* „Internal Clock“.
2. NVIC: Haken bei „TIM2 global interrupt“.
3. Berechnen Sie `PSC` und `ARR` für Update alle 500 ms ($f_{\mathrm{UPD}} = 2\,\mathrm{Hz}$):

$$f_{\mathrm{UPD}} = \frac{f_{\mathrm{TIM}}}{(PSC + 1)\,(ARR + 1)}$$

f_{TIM} aus dem Clock-Tree lesen (APB-Prescaler/TIMPRE beachten) — nicht automatisch `SystemCoreClock`.

4. Generieren Sie den Code. Vor der Schleife: `HAL_TIM_Base_Start_IT(&htim2)` mit Fehlerprüfung. Handler-Kette prüfen: `TIM2_IRQHandler` → `HAL_TIM_IRQHandler` → Callback.
5. Toggle/`HAL_Delay` aus der Main-Schleife entfernen (dort z. B. nur `Heavy_Load()`). Callback **einmal** im vorgesehenen USER-Block (nicht doppelt definieren).

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

1. Hardware-Konfiguration (Beispielrechnung):
f_{TIM} am eigenen Projekt in CubeMX verifizieren. Lehrbeispiel bei $f_{\mathrm{TIM}} = 100\,\mathrm{MHz}$: $PSC = 9999$, $ARR = 4999 \Rightarrow f_{\mathrm{UPD}} = 2\,\mathrm{Hz}$ (Updateabstand 500 ms). LED-Toggle ⇒ vollständiger Blinkzyklus $\approx 1\,\mathrm{Hz}$.

2. Main-Setup und Hintergrundschleife:

```
/* USER CODE BEGIN 2 */
if (HAL_TIM_Base_Start_IT(&htim2) != HAL_OK) {
    Error_Handler();
}
/* USER CODE END 2 */

/* USER CODE BEGIN 3 - in vorhandener while(1) */
Heavy_Load(800000); /* Hintergrund-Last */
/* USER CODE END 3 */
```

3. Interrupt-Callback (einmal, z. B. `USER CODE BEGIN 4`):

```
/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1); /* LD2 gelb */
    }
```

```
}
/* USER CODE END 4 */
```

Fazit: Foreground/Background — die ISR toggelt neben der Main-Last. Unter dieser Konfiguration sind periodische Flanken trotz Hintergrundlast erwartbar; das ersetzt keinen allgemeinen harten Fristnachweis und keine Aussage „Jitter ≈ 0“ ohne Messung.

Aufgabe 4 (Bonus): Drei Messungen am Oszilloskop

Timing wird gemessen, nicht nur behauptet. Pin z. B. **PA0** erst nach Pinbelegung freigeben.

1. **Messung A:** Puls um `Heavy_Load()` (HIGH → Last → LOW). Breite = verstrichene Dauer inkl. möglicher IRQ-Unterbrechungen.
2. **Messung B:** periodische Flanken Main (Aufgabe 2) vs. TIM2-Callback (Aufgabe 3); gleiche Sollperiode; konstante vs. variable Last getrennt untersuchen.
3. **Messung C (optional):** Referenz/Stimulus mit gemeinsamem Zeitbezug vs. früher Handler-Marker; Anfang/Ende nennen (HAL-Callback ≠ reine Hardware-Entry).
4. Trigger, Zeitbasis, Sampleanzahl notieren. Schema: Min/Max/Mittel/Peak-to-Peak — Zahlen erst nach Messung. Formulierung: „gemessene Schwankung unter dieser Konfiguration innerhalb …“, nicht „Jitter ≈ 0“.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4 (Bonus)

Messung A:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET);
Heavy_Load(800000);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_RESET);
```

Erwartete qualitative Aussage (keine erfundenen µs-Werte):

- **A:** Pulsweite = verstrichene Funktionsdauer; bei Präemption durch TIM2 ggf. mit ISR-Anteilen — ggf. zweite Markerspur.
- **B:** Main-Loop zeigt oft größere Periodenschwankung bei variabler Last; TIM2-Pfad ist typischerweise stabiler — die gemessene Peak-to-Peak-Schwankung unter der konkreten Konfiguration dokumentieren.
- **C:** Nur vergleichen, wenn Referenz und Ereignis denselben Zeitbezug haben.

Optimierung, höhere IRQs und Debug-Halt beeinflussen Ergebnisse. Optisch stabiles Blinken ist kein harter Fristnachweis. Messmaximum ≠ sichere WCET-Schranke.